

Slay the Qt 作业报告

队名： 未命名队

组号： 131

项目名称： Slay the Qt

项目类型： 游戏类

开发语言与框架： C++17, Qt Widgets, Qt Graphics View Framework

一、程序功能介绍

项目定位

Slay the Qt 是一个使用 C++ 和 Qt 实现的回合制卡牌构筑游戏。项目参考了 Slay the Spire 的核心玩法：玩家在随机生成的大地图上选择路线，经历普通战斗、精英战、Boss 战、商店、宝箱、篝火和随机事件，通过获得卡牌、金币和遗物逐步构筑牌组，最终挑战 Boss。

主流程

游戏启动后首先进入标题界面，玩家可以开始新游戏或读取存档。进入游戏后，主界面切换到可滚动的大地图。地图按层生成节点，首层固定为普通怪，末层固定为 Boss，中间层随机生成怪物、精英、商店、宝箱、事件和篝火。玩家只能沿着已经连接的路线前进，通关一个节点后才会解锁下一层可选节点。

战斗玩法

战斗系统是项目的核心。每场战斗中，玩家拥有生命、格挡、能量、手牌、抽牌堆、弃牌堆和消耗堆。玩家每回合抽牌并消耗能量打出卡牌，卡牌可以造成伤害、获得格挡、施加状态、抽牌、消耗手牌或提供持续能力。敌人每回合显示意图，之后按槽位顺序行动。战斗胜利后进入奖励界面，玩家可获得金币、卡牌奖励和遗物；失败后删除存档并返回标题界面。

卡牌与遗物

项目已经实现较完整的卡牌和遗物体系。卡牌包含攻击牌、技能牌、能力牌、状态牌和诅咒牌，普通可获得卡牌包括打击、防御、痛击、闪电霹雳、燃烧、金属化、黑暗之拥、薪火之源、剑柄打击、耸肩无视、连续拳、黑暗镣铐、重振精神、死亡收割、壁垒、地狱狂徒和倾泻等。状态与诅咒牌包括黏液、灼伤、伤口、眩晕、疑虑和悔恨。遗物系统实现了 18 件遗物，例如燃烧之血、锚、准备背包、灯笼、开心小花、钢笔尖、手里剑、鸟居、钨合金棒、异蛇之眼和冰淇淋等，覆盖战斗开始、回合开始、回合结束、打出卡牌、伤害修改和战后回复等多种触发时机。

事件与非战斗节点

随机事件和非战斗节点使游戏流程更完整。篝火支持休息回血和锻造升级；商店支持购买卡牌、购买遗物和删除卡牌；宝箱可以获得遗物；问号事件实现了大鱼、蛇影、黏液世界、牧师、尖端设计师、留给自己的讯息和翅膀雕像等事件。问号节点还带有动态概率机制，可能转化为遭遇战、商店或宝箱，避免流程过于固定。

界面与交互

项目使用 Qt Widgets 和 Graphics View 搭建多窗口/多视图体验。主窗口通过堆叠窗口在标题界面、地图、战斗、事件和奖励界面之间切换；战斗界面包含玩家、敌人、手牌、能量、牌堆、状态图标和结束回合按钮；全局顶栏显示生命、金币、牌库数量和遗物栏。项目还加入了黑幕转场、卡牌拖拽、悬停放大、洗牌动画、战利品飞行动画和结尾字幕等表现效果。

二、项目模块与类设计细节

总体架构

项目代码主要位于 src 目录，按职责分为 api、logic、entities、ui 和 map 五个部分。api 层定义跨模块的数据合同和启动器；logic 层负责战斗、卡牌、敌人、遗物和全局存档等逻辑；entities 层定义玩家、敌人、卡牌、遗物和状态等游戏对象；ui 层负责战斗、事件、顶栏、奖励、卡牌显示等图形界面；map 层负责标题界面和大地图。

主控类是 GameWindow。它使用 QStackedWidget 作为界面容器，将标题界面、大地图、战斗视图、事件视图和奖励界面统一管理。GameWindow 还统一负责黑幕转场、顶栏刷新、地图推进、战斗结算、事件结算和存档切换。这样设计后，战斗和事件模块只需要通过信号返回结果，不直接控制全局页面跳转，降低了模块之间的耦合。

数据合同与启动器

BattleAPI.h 中的 BattleContext 和 BattleResult 是战斗模块的输入输出合同。BattleContext 包含玩家当前生命、最大生命、金币、能量、当前卡组、遗物、地图层数和节点类型；BattleResult 包含胜负、战后生命、金币、奖励金币、奖励遗物和卡牌奖励信息。BattleLauncher 只暴露 launch 接口，根据 BattleContext 构造 Player、CardManager、RelicManager、Enemy 列表、BattleEngine 和 BattleView，并在战斗结束时发出 BattleResult。

EventAPI.h 中的 EventContext 和 EventResult 是事件模块的输入输出合同。EventContext 描述进入事件前的玩家快照、牌组、遗物和事件类型；EventResult 描述事件后的生命、最大生命、金币、牌组和遗物变化。EventLauncher 根据事件子类型创建对应事件视图，并在事件结束时统一回收沙盒状态，返回给 GameWindow。问号遭遇战则通过 BattleLauncher 转入战斗流程。

核心战斗逻辑

BattleEngine 是战斗状态机，负责回合流程、出牌校验、敌人行动、胜负判断和各种钩子触发。战斗开始时，BattleEngine 调用敌人的 onBattleStart 和遗物的战斗开始钩子，然后进入玩家回合。玩家回合开始时，所有存活敌人先 rollNextIntent，随后引擎刷新意图伤害，重置或保留格挡，处理能量、遗物和能力状态，再按玩家基础抽牌数抽牌。

玩家出牌时，BattleEngine 先检查卡牌是否还在手牌中，再检查能量是否足够，然后调用卡牌的 play 方法执行效果。卡牌结算后，RelicManager 广播 onCardPlayed，手牌动态文案刷新。能力牌进入能力区，带消耗标记的牌进入消耗堆，普通牌进入弃牌堆。敌方全灭后，引擎触发战后遗物效果，并延迟发出 battleEnded 信号，让 UI 动画有时间播放。

敌人回合由 processEnemyTurn 和 processNextEnemyAction 实现。敌人按照 slotIndex 从左到右依次行动，支持攻击、防御、施加状态、召唤、群体强化、群体叠甲、攻击并塞入状态牌等多种意图。新召唤的敌人先进入 pendingSpawns，当前轮行动完成后再加入战场，避免遍历列表时修改导致错误。

CardManager 管理战斗中的抽牌堆、手牌、弃牌堆、消耗堆和能力区。它负责初始化卡组、洗牌、抽牌、回合结束弃牌、消耗、强制移入弃牌或消耗、永久删除卡牌和随机升级卡牌。CardManager 通过 cardDrawn、cardDiscarded、cardExhausted、pileCountsChanged、deckShuffled 等信号通知 UI，同步逻辑和表现。

游戏对象与多态设计

Fighter 是 Player 和 Enemy 的共同基类，封装名称、生命、最大生命、格挡、死亡状态和 StatusManager。Player 在 Fighter 基础上扩展能量、金币、基础抽牌数以及加能量、扣血、改金币等接口。Enemy 在 Fighter 基础上扩展敌人图片、敌人 ID、槽位、当前意图和行动历史，并通过虚函数 rollNextIntent 让不同敌人拥有不同 AI。

Card 是所有卡牌的抽象基类，包含 ID、名称、费用、描述、图片、类型、目标、稀有度、是否虚无、是否消耗、是否升级等属性。每张具体卡牌继承 Card 并重写 play 和 upgrade。例如打击造成伤害，防御获得格挡，痛击造成伤害并施加易伤，燃烧给玩家力量，金属化提供回合结束格挡，黑暗之拥让消耗触发抽牌，倾泻根据当前能量自动打出牌库顶牌。通过基类接口，BattleEngine 不需要知道每张牌的具体类型，只需要调用 play。

Relic 是所有遗物的基类，提供 onBattleStart、onBattleEnd、onCardPlayed、modifyAttackDamage、modifyBl

ock、onTurnStart、onTurnEnd 和 modifyIncomingDamage 等钩子。RelicManager 保存当前遗物列表，并在对应时机顺序广播或叠加修改数值。这样新增遗物时只需要继承 Relic 并重写相关钩子，战斗主流程不需要改动。

StatusManager 使用 QMap 保存各种状态层数，并提供 calculateDamage 和 calculateBlock 两个静态计算函数。伤害计算会考虑力量、虚弱和易伤；格挡计算会考虑敏捷和脆弱；回合结束时会递减虚弱、易伤、脆弱，并处理镣铐和仪式等特殊状态。状态变化通过 statusChanged 信号通知 UI 刷新状态图标。

工厂与资源组织

CardFactory 根据卡牌 ID 创建具体 Card 对象，也提供随机卡牌池和三选一卡牌奖励生成。RelicFactory 根据遗物 ID 创建具体 Relic 对象，并从未拥有遗物中随机掉落。EnemyFactory 根据节点类型和层数生成战斗编队，例如普通怪池、精英怪池和 Boss 池，再按 encounterId 组装具体敌人。工厂集中管理 ID 到类的映射，使存档、奖励、商店和事件只需要传递字符串 ID。

资源文件集中放在 resources 目录，并通过 resources.qrc 注册。卡牌、遗物、敌人、地图节点、事件背景和 UI 图标都通过 Qt 资源路径加载。这样程序发布时不依赖零散外部路径，减少资源丢失风险。

地图、事件和界面模块

MapManager 负责随机生成 15 层地图。每层生成 2 到 4 个节点，末层固定 Boss，首层固定普通怪，第 8 层固定篝火，其余层按概率生成怪物、精英、商店、宝箱、事件和篝火。MapNode 保存节点 ID、层数、位置、节点类型、后继节点和 UI 坐标。MapManager 绘制地图背景和层间连线，并用按钮表示节点。访问过的节点会标记并禁用，未解锁节点也不能点击。

事件 UI 采用 EventBaseView 和 GenericChoiceEventView 复用公共结构。EventBaseView 负责场景、玩家图片、离开按钮和遮罩层；GenericChoiceEventView 提供统一背景、事件图片、标题、可滚动文本和选项按钮。具体事件类只配置标题、描述、图片和选项回调。例如大鱼事件提供回血、加最大生命和遗物加诅咒三种选择；牧师提供治疗和删牌；设计师提供升级、删除、组合服务和一拳离开。

BattleView 是战斗表现层，绑定 BattleEngine 后创建玩家、敌人、手牌布局、牌堆、能量球、结束回合按钮和确认按钮。CardItem、EnemyItem、PlayerItem、PileItem、EnergyWidget、StatusIconItem 等 QGraphicsObject 负责具体图形绘制和交互。CardItem 支持拖拽、悬停、可打出状态、选择高亮、价格显示和播放弃牌/消耗动画，因此同一个卡牌组件可以复用于战斗、商店、篝火和事件选牌。

GlobalSaveData 是全局存档单例，保存当前生命、最大生命、金币、最大能量、删牌费用、事件抽奖袋、问号怪物概率、牌组 ID 列表和遗物 ID 列表。存档写入 savegame.json，只保存稳定 ID 而不是对象指针。读档时再通过工厂重建对象，避免跨场景指针失效。

三、小组成员分工情况

成员分工

成员	主要分工
王文韬	项目策划；战斗系统；卡牌、遗物、状态、敌人和战斗结算逻辑；卡牌与遗物工厂；核心战斗 UI 联调。
章家玮	事件系统；篝火、商店、宝箱和问号事件；事件通用 UI；事件与全局存档、顶栏、卡牌选择和遗物奖励的联调。
齐宇豪	标题界面、大地图和主流程；地图节点生成、路线限制、滚动交互；GameWindow 页面切换和黑幕转场的整体集成。

四、项目总结与反思

完成情况

本项目完成了课程要求中的 C++ 面向对象设计和 Qt 图形界面要求。程序不是单一窗口，而是包含标题界面、大地图、战斗界面、事件界面、商店、宝箱、篝火、奖励界面和结尾字幕等多个视图。代码结构上，项目把实体、逻辑、接口合同和 UI 分层，并大量使用继承、多态、信号槽和工厂方法，能比较自然地扩展新卡牌、新遗物、新敌人和新事件。

架构收获

项目最成功的部分是战斗机制的可扩展性。卡牌效果放在 Card 子类中，遗物效果放在 Relic 钩子中，状态计算放在 StatusManager 中，战斗主流程只负责调度。这样新增一张卡或一个遗物时，不需要在 BattleEngine 中写大量分支。事件模块也通过 EventContext 和 EventResult 做了沙盒结算，避免事件 UI 直接修改所有全局对象。

状态同步问题

开发中的主要难点来自状态同步。战斗和事件都需要同时更新逻辑数据、顶栏、地图进度、牌堆数量、遗物栏和存档。如果每个模块都直接修改全局状态，容易出现 UI 和实际数据不一致。项目后期通过 GlobalSaveData、BattleResult、EventResult 和 GameWindow 统一回写，减少了这类问题。

生命周期管理

另一个难点是 Qt 图形界面的生命周期管理。战斗视图、事件视图、卡牌图元、遗物对象和动画对象经常跨信号槽传递，如果父子关系或 deleteLater 处理不当，容易造成悬空指针或重复释放。项目中大量使用 Qt 父子树管理对象，并在返回地图时移除和销毁当前视图，基本保证了页面切换的稳定性。

项目缺陷与改进方向

1. 创新性不足

问题：整体玩法、卡牌构筑、地图节点、遗物触发和大部分事件设计都明显参考了成熟作品 Slay the Spire。虽然项目在 Qt 实现、模块拆分、事件沙盒、部分卡牌和表现效果上做了自己的整合，但从玩法体验来看，更像是对经典机制的复现和工程化练习，原创机制和独立主题表达还不充分。

改进方向：后续可以加入更贴合“Qt”或课程特色的机制，例如与程序设计概念相关的卡牌、调试器主题事件、代码资源管理类遗物，或让玩家在战斗中利用面向对象、继承、多态等概念形成独有策略，使项目不只是复刻玩法，而是形成自己的主题表达。

2. 工厂映射维护成本较高

问题：CardFactory、RelicFactory 和 EnemyFactory 仍然使用显式 if/else 维护 ID 到类的映射。当前内容规模下可以接受，但新增卡牌、遗物或敌人时需要修改工厂文件，长期维护容易遗漏注册或写错 ID。

改进方向：如果内容继续增加，可以考虑注册表式工厂，把 ID、构造函数和基础元数据集中登记，减少重复分支。这样新增内容时只需要补充注册项，也方便后续制作图鉴、调试面板或内容检查工具。

3. 战斗反馈与操作引导不足

问题：战斗系统已经支持卡牌、状态、遗物和敌人行为，但部分机制仍偏依赖玩家阅读文字和数值变化。例如状态层数变化、遗物触发时机、敌人意图含义和事件后果没有形成足够醒目的统一反馈，新玩家可能需要通过反复试错理解规则。

改进方向：后续可以补充统一的提示层和反馈动画，在状态图标、遗物触发、伤害与格挡变化、敌人行动预告等位置给出更明确的视觉提示；关键事件选项也可以显示收益和风险，使玩家更容易理解当前决策的影响。

4. 数值平衡仍需打磨

问题：地图节点概率、敌人池、卡牌费用、遗物掉落和奖励金币等数值目前主要依据经验设定，缺少足够多的试

玩数据支撑。部分组合可能过强或过弱，影响长期游玩体验。

改进方向：后续可以记录每局路线、胜率、常用卡牌、遗物触发频率和平均战斗回合数，根据数据调整掉落概率、敌人强度和奖励数值，使游戏难度曲线更加平滑。

总体评价

总体而言，Slay the Qt 已经实现了一个完整的可运行游戏闭环：开始游戏、选择路线、进入战斗或事件、获得奖励、更新牌组和遗物、继续爬塔并挑战 Boss。项目在有限课程周期内完成了较多玩法系统，也体现了 C++ 面向对象和 Qt 信号槽在中型 GUI 游戏项目中的组织方式。